



Proyecto 2. Valor 2%

Temas de Estudio

1. Tema 1: Herencia, polimorfismo e interfaces.
2. Tema 2: Interfaces gráficas de usuario y manejo de archivos (flujos E/S).

Objetivo

Desarrollar una aplicación Java con interfaz gráfica (GUI) que permita gestionar reportes de **mascotas perdidas y encontradas**. El proyecto busca aplicar los conceptos de herencia y polimorfismo para representar distintos tipos de mascotas, además de implementar la persistencia de datos en archivos

La aplicación incluirá un módulo visual de coincidencias, que sugerirá posibles correspondencias entre reportes de pérdidas y hallazgos, con el fin de facilitar el reencuentro de las mascotas con sus dueños. Este proyecto representa la segunda etapa del sistema desarrollado en el Proyecto #1, evolucionando de una aplicación en consola hacia una con interfaz gráfica.

Software de desarrollo

NetBeans

Con el fin de mantener claridad en el desarrollo, el proyecto deberá compilarse y ejecutarse utilizando Java **Ant**, que es un enfoque tradicional y sencillo de construcción en NetBeans. **No obstante, también se permitirá el uso de Maven**, siempre que la entrega incluya todas las dependencias necesarias y el proyecto sea ejecutable sin configuraciones adicionales.

Desarrollo

El refugio “Huellas Felices” administra un registro de mascotas perdidas y encontradas. En esta segunda etapa, se busca mejorar el sistema implementado previamente, evolucionando de una aplicación en consola a una aplicación con interfaz gráfica (GUI).

La aplicación se llamará Gestor de Reportes de Mascotas GUI y tendrá como finalidad apoyar la localización y recuperación de mascotas mediante un registro más completo y una visualización más amigable para el usuario. En esta versión, el sistema deberá ofrecer como mínimo las siguientes funcionalidades. Estas deben implementarse con interfaz gráfica, pero manteniendo la lógica planteada en el Proyecto 1.

En esta versión, el sistema deberá ofrecer como mínimo las siguientes funcionalidades, manteniendo **todas las características del Proyecto 1** (menús, validaciones, reportes y coincidencias) pero ahora implementadas con **interfaz gráfica** y extendidas con los nuevos requerimientos:

- Capturar datos de la persona que reporta y de la mascota, validando todos los campos definidos en el Proyecto 1 (incluyendo unicidad de Id de Reporte, formato de cédula, mínimo de caracteres y todas las demás validaciones).
- Consultar reportes registrados según criterios: **ID del reportante, especie o zona**.
- Generar reportes: **general, agrupado y de coincidencias**, en forma tabular dentro de la interfaz gráfica.
- Actualizar un reporte, permitiendo editar un solo dato o reingresar todos los datos, excepto el Id de Reporte, el Id del Reportante y la Fecha del reporte.
- Detectar y mostrar coincidencias visuales entre reportes de pérdidas (PDR) y hallazgos (ENC), resaltando coincidencias prioritarias (microchip o múltiples criterios coincidentes).
- **Mantener persistencia de datos mediante archivos de texto (.txt)**, sin utilizar serialización de objetos. El objetivo es que el estudiante demuestre su capacidad para **leer, validar, parsear y escribir** datos en archivos planos, conservando los registros entre ejecuciones.
 - **No se permite** el uso de serialización de objetos (ObjectOutputStream, Serializable) ni formatos binarios (.dat).

Los datos deben almacenarse y recuperarse desde **archivos de texto plano** usando **flujos de E/S** (por ejemplo, FileReader, BufferedReader, FileWriter, BufferedWriter, PrintWriter).

- Aplicar herencia y polimorfismo en la representación de mascotas y reportes, extendiendo el modelo de clases definido en el Proyecto 1.

Este proyecto refuerza la aplicación de los principios de Programación Orientada a Objetos (POO), incluyendo herencia y polimorfismo, además del desarrollo de interfaces gráficas y el manejo de archivos.

Datos que se deben solicitar al usuario

En la interfaz gráfica, los formularios de registro y edición deberán solicitar los siguientes datos, aplicando las mismas validaciones definidas en el Proyecto 1. Cada campo se implementará con los componentes de Swing más apropiados para garantizar facilidad de uso y consistencia en el formato.

- **Id de Reporte:** identificador único ingresado por el usuario, con el prefijo REP- seguido de **cuatro dígitos** (ej.: REP-0001, REP-0002). **Debe validarse la unicidad;** si ya existe, se debe solicitar uno nuevo.
- **ID reportante:** cédula costarricense en el formato 1-1111-1111. (Puede repetirse en distintos reportes).
- **Nombre completo:** mínimo 7 caracteres; se permiten espacios.
- **Tipo de Reporte (3 letras):** PDR = Pérdida, ENC = Encontrada (en mayúsculas). Por defecto PDR si no se indica valor.
- **Fecha del reporte:** se obtiene automáticamente del sistema. Opcionalmente se admite ingreso manual en formato dd/MM/yyyy; si se deja en blanco, se asigna la fecha actual.
- **Zona:** país, provincia, cantón o distrito; texto libre, **máximo 30 caracteres**.
- **Especie (3 letras):** DOG = Perro, CAT = Gato (en **mayúsculas**)
- **Color principal y señas particulares** texto libre, mínimo 10 caracteres
- Teléfono: #####-#####.
- **Microchip** Código alfanumérico; si se desconoce, dejar en blanco (opcional).

Notas importantes:

- Los códigos PDR, ENC, DOG, CAT deben ingresarse **exactamente en mayúsculas**.
- **Ningún campo obligatorio** puede quedar en blanco.
- Validar **mínimos/máximos:** Nombre ≥ 7 , Señas ≥ 10 , Zona ≤ 30 .
- La fecha manual debe cumplir estrictamente dd/MM/yyyy.
- Los mensajes de error **deben mostrarse en la GUI** (no en consola).

Modelo POO (Herencia y Polimorfismo)

El sistema debe implementarse utilizando los principios de Programación Orientada a Objetos (POO), aplicando herencia y polimorfismo para representar las mascotas y los reportes.

Modelo de Mascotas

Clase Mascota: contendrá los campos comunes a todas las mascotas:

- especie (DOG/CAT)
- color
- señas

- microchip (opcional)

Además, declarará un método `resumenMascota()`, que cada subclase implementará para mostrar la información de manera legible.

Subclases:

- Perro: heredará de `Mascota` y podrá incluir atributos específicos como raza, talla (PEQ/MED/GRA) o collar (booleano).
- Gato: heredará de `Mascota` y podrá incluir atributos específicos como raza, tipoPelaje (CORTO/LARGO) o esterilizado (booleano).

Modelo de Reportes:

- Clase base `ReporteMascota`: contendrá los datos generales del reporte y una relación con la mascota reportada.
- `idReporte` (formato REP-0001, único)
- `idReportante` (cédula 1-1111-1111)
- nombre (mínimo 7 caracteres)
- tipo (PDR o ENC)
- fecha (automática o ingresada en formato dd/MM/yyyy)
- zona (máximo 30 caracteres)
- telefono (formato ####-####)
- mascota (objeto de tipo `Mascota` → aquí se incluyen especie, color, señas y microchip)

Subclases:

- `ReportePerdida`: fija el tipo en PDR.
- `ReporteEncontrada`: fija el tipo en ENC.

Persistencia en archivo (.txt)

- La información de los reportes deberá conservarse en archivos de texto plano. Existirán dos archivos separados: `ReportesPerdidas.txt` para los reportes de tipo PDR y `ReportesEncontradas.txt` para los de tipo ENC.
- Si un reporte cambia de tipo (de PDR a ENC o viceversa), el registro debe moverse al archivo correspondiente, manteniendo la unicidad del `idReporte` y sin eliminar el resto de registros.
- El sistema debe ser capaz de cargar los datos al iniciar, así como agregar y actualizar, validando siempre la unicidad del `idReporte` y los formatos indicados (códigos, cédula, teléfono, fecha y todos los demás.).
- No se permite el uso de serialización ni archivos binarios. La manipulación de los datos debe hacerse con flujos de texto (lectura y escritura .txt), de forma que el estudiante demuestre su capacidad para validar, parsear y

manejar archivos planos.

Reportes del sistema

Los reportes del Proyecto 2 son los mismos del Proyecto 1, pero ahora deben mostrarse en la **interfaz gráfica** mediante tablas con columnas claras.

Reporte general

Muestra la lista completa de reportes registrados en el sistema. Cada fila corresponde a un reporte y debe incluir como mínimo:

- ID reportante
- Nombre completo
- Fecha del reporte
- Zona
- Tipo (PDR/ENC)

Este reporte permite al refugio tener una visión global de todos los casos activos en los archivos de pérdidas y hallazgos.

El resultado debe mostrarse en forma tabular dentro de la GUI y actualizarse cada vez que se agreguen, modifiquen registros, leerse directamente del archivo **ReportesPerdidas.txt** y **ReportesEncontradas.txt**.

Reporte agrupado

Presenta un conteo de reportes según dos criterios:

Por tipo: cantidad de reportes de pérdida (PDR) y de hallazgo (ENC).

Por especie: cantidad de reportes por especie (DOG y CAT).

El resultado debe mostrarse en forma tabular dentro de la GUI y actualizarse cada vez que se agreguen, modifiquen registros, leerse directamente del archivo **ReportesPerdidas.txt** y **ReportesEncontradas.txt**.

Reporte de coincidencias (visual)

La aplicación detectará y mostrará coincidencias entre reportes de pérdidas (PDR) y hallazgos (ENC) en un componente tipo Jtable, en una ventana dedicado para ello dentro de la GUI.

Cuando se calcula: al **registrar** o **actualizar** un reporte y cuando el usuario lo solicite desde la GUI.

Propósito de la comparación

El refugio *Huellas Felices* tiene dos tipos de reportes:

- **PDR (Pérdida):** cuando alguien reporta que su mascota se perdió.
- **ENC (Encontrada):** cuando alguien reporta que halló una mascota.

El objetivo del **módulo de coincidencias** es **cruzar automáticamente** estos dos tipos de reportes para detectar **pares que podrían referirse a la misma mascota**.

Esto ayuda a **acercar al dueño y a la persona que encontró la mascota**, facilitando el reencuentro.

Los criterios de coincidencia son los mismos definidos en el Proyecto 1:

Criterio	Regla para coincidencia
Especie	Debe coincidir (DOG/CAT)
Color	Coincidencia exacta
Zona	Coincidencia exacta
Fecha	Diferencia menor o igual a 7 días entre PDR y ENC
Microchip	Si coincide, marcar como coincidencia prioritaria

Ejemplo

Id PDR	Id ENC	Nombre PDR	Nombre ENC	Fecha PDR	Fecha ENC	Zona	Especie	Color	Microchip	Prioridad
REP-0003	REP-0010	Juan Pérez López	Ana Gómez Torres	05/08/25	06/08/25	San José	DOG	Marrón	123ABC456	PRIORIT.
REP-0004	REP-0012	Luis Rodríguez	Pedro Morales	07/08/25	08/08/25	Heredia	CAT	Negro	—	Normal
Total coincidencias: 2										

- **PDR:** Juan reporta un perro marrón perdido en San José el 05/08/2025.
- **ENC:** Ana reporta un perro marrón encontrado en San José el 06/08/2025.

El sistema detecta coincidencia → porque coincide especie, color, zona y la fecha está dentro de 7 días.

Además, si ambos registros tienen el mismo microchip, se marca como **PRIORITARIA**.

Cada vez que se registra o actualiza un reporte:

- El sistema toma todos los **PDR** y todos los **ENC de los archivos txt**.
- Los compara con base en los criterios del enunciado:
- **Especie:** debe ser la misma (DOG/CAT).
- **Color:** coincidencia exacta.
- **Zona:** coincidencia exacta.
- **Fecha:** diferencia de **≤ 7 días** entre la pérdida y el hallazgo.
- **Microchip (si existe):** si coincide, es **prioritario**.
- Si se cumplen esos criterios, el sistema arma un **par PDR-ENC**.
- Todos los pares detectados se muestran en **una sola tabla** dentro de la GUI.

Cada vez que se registra o actualiza un reporte, el sistema compara automáticamente los archivos **ReportesPerdidas.txt** y **ReportesEncontradas.txt** y muestra todas las coincidencias encontradas en una tabla tabular dentro de la GUI.

Importante: Cuando un reporte coincide con varios del otro tipo, el sistema no los agrupa en un único resultado, sino que genera un par independiente para cada combinación válida.

Por ejemplo:

- 1 registro ENC puede coincidir con 4 registros PDR.
- El sistema mostrará en la tabla cuatro filas diferentes, una por cada par detectado:
- ENC(1)–PDR(1), ENC(1)–PDR(2), ENC(1)–PDR(3), ENC(1)–PDR(4).

De esta forma, se garantiza que todas las posibles coincidencias se visualicen claramente en la GUI, sin perder información relevante para el refugio.

Interfaz gráfica (Swing)

Es de total importancia que el estudiante lea esta sección y se apegue a ella, cualquier incumplimiento será penalizado con nota de 0.

- La aplicación debe desarrollarse en **NetBeans** utilizando exclusivamente **Java Swing**, que es la tecnología oficial del curso. No se permite el uso de JavaFX o herramienta de trabajo, cualquier entrega fuera de este lineamiento será calificada con **nota de 0**.
- Cuando se detecten **coincidencias evidentes con estructuras de código generadas automáticamente por IA, sin adaptación ni justificación por parte del estudiante**. Esto se debe a que el objetivo del curso es que el estudiante aprenda a analizar problemas, diseñar soluciones y codificarlas siguiendo principios de Java. Copiar sin entender impide que el alumno desarrolle la lógica necesaria para programar y, además, los proyectos generados por IA suelen presentar estructuras genéricas, con errores ocultos o estilos de codificación que no cumplen con las normas enseñadas en clase.
- Es obligatorio que la interfaz se construya utilizando la **herramienta gráfica de NetBeans para arrastrar y soltar componentes** (GUI Builder). No se aceptarán códigos escritos manualmente desde cero para generar la interfaz gráfica. La creación “100% por código” de los formularios será calificada con **nota de 0**, ya que el objetivo es que el estudiante se familiarice con la herramienta oficial y el proceso de construcción visual.
- El uso de **Inteligencia Artificial** para generar formularios completos o código desde cero no está permitido. En los últimos años se ha detectado que algunos estudiantes utilizan IA para producir ventanas con código directo en vez de usar el editor gráfico. Esto invalida el aprendizaje esperado y cualquier entrega en estas condiciones será evaluada con **nota de 0**. Todos los formularios deben ser diseñados con la herramienta interna de NetBeans, respetando los principios del curso.

La finalidad es evaluar la capacidad del estudiante para **analizar problemas, diseñar soluciones y programar de manera ordenada en Java**, aplicando herencia, polimorfismo y modularidad. Un sistema que funcione, pero esté mal estructurado

no cumple con los objetivos de aprendizaje planteados.

Requisitos adicionales y validaciones

Indicaciones de interfaz (estrictas): El proyecto se desarrollará exclusivamente con Java Swing y únicamente usando el GUI Builder de NetBeans (Palette).

Todos los componentes visuales deben arrastrarse desde la Palette **(sin crear componentes gráficos por código “desde cero”)**.

Nota: Esta restricción se establece con el fin de disminuir el uso de Inteligencia Artificial en la creación de proyectos, ya que las herramientas de IA suelen generar componentes gráficos directamente por código. Al limitar el desarrollo a componentes arrastrados desde la Palette, se fomenta que el estudiante adquiera práctica real en el uso de NetBeans y en la construcción manual de interfaces gráficas.

Cualquier proyecto que no cumpla con estas indicaciones, o que incluya código de componentes gráficos creado manualmente, será calificado con nota 0 de forma automática.

- Prohibido usar librerías externas de UI o código generado por IA para construir interfaces.
- La personalización visual se hará mediante propiedades del diseñador (propiedad, Layout, eventos) y no con código manual que reemplace al diseñador.
- Proyecto **modular** (clases separadas, responsabilidades claras, métodos pequeños).
- Documentación mínima: comentario de clase y de métodos (qué hace, parámetros y retorno).
- No se aceptan **coincidencias parciales** (color/zona exactos; especie exacta; fecha ≤ 7 días). **Prioritario** si microchip coincide; mostrar primero en la tabla.
- Validar antes de guardar; errores por **JOptionPane** (no consola).
- Capturar excepciones específicas (p. ej., `NumberFormatException`, `IOException`) y mostrar mensajes **claros** con acción sugerida.
- Al cambiar de **tipo** (PDR ↔ ENC), **mover** el registro entre archivos manteniendo el mismo `idReporte`.
- Códigos (PDR/ENC/DOG/CAT) se **guardan en mayúsculas**.
- **Unicidad de idReporte** verificada al **agregar/actualizar** (revisar ambos archivos).
- Especie solo DOG o CAT; Tipo solo PDR o ENC.

Honestidad Académica



<https://audiovisuales.uned.ac.cr/play/player/23048>

Nota importante

- Cada estudiante es responsable del contenido que entrega.
- Si el archivo enviado no corresponde al proyecto correcto, no se aceptarán entregas posteriores fuera de la fecha establecida.
- Si se detecta que el contenido del archivo coincide con otro estudiante o no es de su autoría, se aplicarán las sanciones indicadas en el documento de Lineamientos ante casos de plagio disponible en la plataforma.

Indicaciones Importantes

- Es obligatorio que incluya todo el directorio donde se encuentra Proyecto 2.
 - **El Proyecto 2** debe estar desarrollado en NetBeans que es la herramienta oficial del curso.
 - El programa debe ser modular, utilizando de la mejor manera funciones definidas por usted.
 - Los trabajos deben realizarse en forma individual. Dentro del código del programa debe de indicar la documentación que explique cómo fue realizado el programa.
 - Si utiliza código de algún ejemplo del libro, o de otra fuente que no sea de su autoría, debe de indicarlo.
 - Comprima todos los archivos en un solo archivo .zip o .rar.
 - Nombre del archivo que envía: debe ser nombre y primer apellido del estudiante, y nombre de la tarea. Ejemplo: JuanRojas-proyecto2.
 - La entrega del **Proyecto 2** en las fechas establecidas en la plataforma de aprendizaje en línea Moodle en el apartado que se indique.
 - Si no concluyó a tiempo la tarea, debe entregar lo que pudo hacer e incluir una carta explicando las razones por las cuales no finalizó.

Rúbrica de Evaluación

Criterio	Alto	Medio	Bajo	No cumple
- La GUI no está hecha con Swing o no se construyó con el GUI Builder de NetBeans (p. ej., JavaFX, formularios “100% por código”, librerías externas). - Persistencia no realizada con archivos .txt (uso de .dat, serialización u otro formato binario). - Uso de IA para generar formularios/código completo en lugar de usar el GUI Builder, estructuras similares. - Si se incurre en cualquiera de estos puntos, la calificación del proyecto es 0 independientemente de otros rubros.	10	5	3	0
Estructura y organización del código Qué se evalúa: Diseño modular y limpio. - Clases separadas por responsabilidad, métodos cortos, nombres adecuados en convención Java, paquetes organizados, sin mezclar lógica de negocio en la GUI. - Estructura aceptable, aunque con métodos largos o clases que combinan varias funciones. - Proyecto funcional pero desordenado, con responsabilidades mezcladas.	8	6	3	0
Modelo POO (atributos y constructores en Mascota/ReporteMascota) Qué se evalúa: implementación fiel del modelo base. - Clase Mascota con atributos comunes (especie, color, señas, microchip) y método abstracto/polimórfico. Clase ReporteMascota con todos los atributos definidos en el enunciado, constructores completos y encapsulación. - Modelo implementado, pero con atributos faltantes o constructores incompletos. - Clases creadas sin respetar todos los atributos o sin constructores adecuados.	5	3	1	0
Herencia aplicada. Qué se evalúa: uso correcto de herencia para extender el modelo. Subclases Perro y Gato con atributos	8	6	3	0

específicos; ReportePerdida y ReporteEncontrada que fijan el tipo en PDR/ENC. • Subclases presentes pero con atributos mínimos o sin diferenciar bien sus responsabilidades. • Herencia implementada de forma superficial, sin aportar nada al modelo.				
Polimorfismo y encapsulación (uso real de override, getters/setters adecuados, atributos privados, método polimórfico). Qué se evalúa: uso real de polimorfismo y respeto al encapsulamiento. • Implementación de resumenMascota() polimórfico, uso de override, getters/setters adecuados, atributos privados. • Uso parcial de polimorfismo o encapsulación incompleta. • Sobrescritura mínima o getters/setters mal aplicados.	8	6	3	0
Interfaz gráfica Swing en NetBeans (GUI Builder) Qué se evalúa: interfaz creada con GUI Builder y flujo completo de uso. • Formularios hechos con GUI Builder, flujo funcional (registrar, consultar, actualizar, reportes y coincidencias), tablas que se refrescan tras cambios y mensajes con JOptionPane. • GUI con la mayoría de las funciones, pero con algunos errores de flujo o refresco. • GUI incompleta o que delega lógica en consola.	8	6	3	0
Persistencia en archivos .txt y movimiento PDR↔ENC Qué se evalúa: gestión de archivos de texto. • Carga inicial desde ReportesPerdidas.txt y ReportesEncontradas.txt, operaciones de actualización seguras, movimiento de registros entre archivos al cambiar tipo, manejo de IOException y errores de línea. • Persistencia funcional pero con errores en reescritura o movimiento de registros. • Se guardan datos pero sin respetar unicidad o con riesgo de duplicados.	15	10	5	0
Validaciones y manejo de excepciones	10	5	3	0

Qué se evalúa: validaciones completas y uso de excepciones específicas. • Regex aplicadas (cédula, teléfono, códigos PDR/ENC/DOG/CAT en mayúsculas), nombre ≥ 7, señas ≥ 10, zona ≤ 30, fecha dd/MM/yyyy validada, unicidad de idReporte en ambos archivos. Manejo de IOException, NumberFormatException, etc., con mensajes en JOptionPane. • Validaciones presentes pero incompletas. • Validaciones mínimas o genéricas.				
Reporte General en GUI • Lista completa de reportes (ID reportante, Nombre, Fecha, Zona, Tipo) leída de los archivos, refrescada tras cambios. • Reporte generado pero con columnas incompletas o sin refresco. • Reporte parcial o con datos incorrectos.	5	3	1	0
Reporte Agrupado en GUI • Conteos correctos por Tipo (PDR/ENC) y por Especie (DOG/CAT), refrescados tras cambios. • Conteos presentes pero con errores de cálculo o refresco. • Solo un criterio de agrupación o incompleto.	10	7	4	0
Reporte de coincidencias • Detecta coincidencias según especie, color, zona, fecha ≤ 7 días; prioridad si coincide microchip; muestra resultados en tabla dentro de la GUI. • Coincidencias detectadas parcialmente (faltan criterios o prioridad). • Coincidencias muy básicas o mal calculadas.	10	7	4	0
Comentarios y documentación mínima Qué se evalúa: documentación clara en todas las clases. • Todas las clases y métodos comentados (qué hace, parámetros/retorno si aplica), más notas en lógica no trivial (parseo .txt, coincidencias). • Comentarios en la mayoría de clases, pero faltan en algunos métodos. • Comentarios muy escasos.	3	2	1	0
TOTAL	100			